

Console UX Redesign — Fidelity-First Product-Development Platform

Status: proposal · **Owner:** product · **Supersedes:** the per-cloud "mimic the native console" approach

1. Thesis

Vyomi is not a prettier cloud console. It is **the safe, free, reproducible place to *build and run* real cloud products — faithful cloud APIs backed by real infrastructure — then ship the same code to prod.**

We stop competing on console fidelity (an unwinnable war against the hyperscalers' consoles) and compete on what the real cloud structurally *cannot* be: **instant, transparent, reproducible, offline-capable, multi-cloud, and \$0-risk.**

North-star sentence:

Develop against a faithful cloud API **backed by real infrastructure you can SSH into, store real data in, and run real queries against** — build the actual product here, then ship the same code to prod.

2. The two planes of fidelity (the core model)

Every design decision serves one of two planes. The UI must make **both** visible.

Plane	Promise	Proven in the UI by
Control plane (cloud API)	boto3 / gcloud / az / Terraform behave like the real cloud, unchanged	Conformance panel + glass-box call inspector
Data plane (real substrate)	behind the API a real backend runs you can build on — LXD/Docker compute (real SSH), MinIO (real objects), MySQL/Postgres (real SQL), Vault, NATS	Connect-to-the-real-backend surfaces + call→backend links

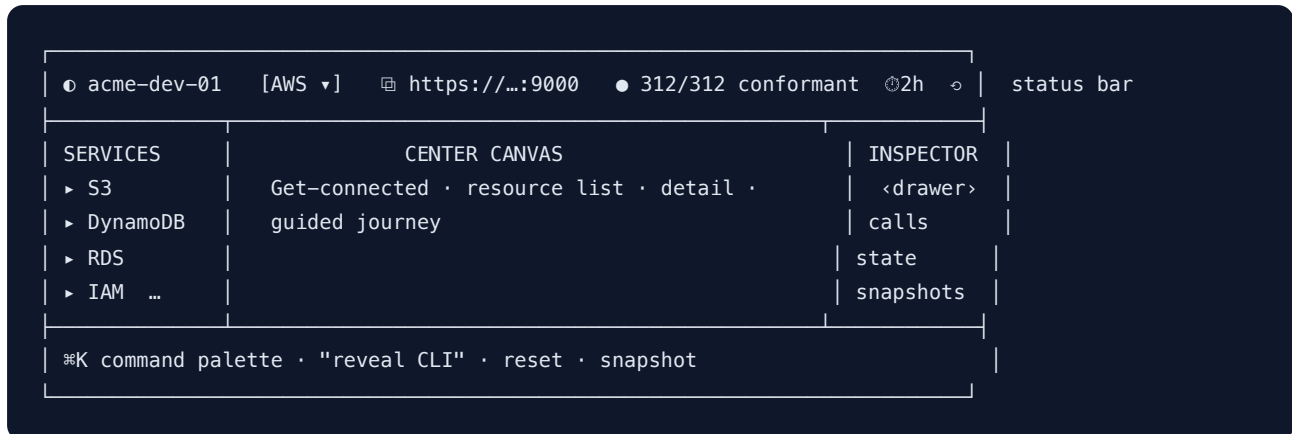
The data plane is the differentiator. Mock-only tools can fake the control plane; they cannot let you SSH into a box and deploy an app, or run real SQL that persists.

3. Product tenets (non-negotiable)

- Endpoint-first, console-second.** The connection is the hero; the console is an inspection/build lens. Product dev happens in the IDE/CLI/CI.
- Fidelity is always on screen.** A conformance signal is persistent; the fake proves itself continuously, never just claims it.

3. **The glass-box is one keystroke away.** Every API call is inspectable and linked to the real backend that served it.
4. **State is version-controlled.** Snapshot / fork / rollback / replay are first-class — over *real* backend state (MinIO buckets, SQL data, container state).
5. **One shell, every substrate & cloud.** Local `vyomi up`, Nano (browser), and the Codespaces sandbox share the identical journey; AWS/GCP/Azure are lenses.
6. **Recognizable, not cloned** (*the anti-mimicry tenet*). See §7.

4. The persistent shell (the "common UI")



- **Status bar:** workspace · cloud-lens switcher · endpoint (click-copy) · conformance pill · TTL/cost chip · reset.
- **Left rail:** resource explorer (only the profile's services), searchable.
- **Center:** active view; starts as *Get connected*.
- **Inspector drawer (⌘I):** the glass-box — Calls / State / Snapshots / Conformance.
- **Command palette (⌘K):** do anything, jump anywhere, reveal the CLI for any action.

5. The journey — three chords + a build-&run arc

Moment 1 — `vyomi up` → running faithful cloud in seconds (*chord: speed*)

Progressive readiness (Wave-1 console/API instant; Wave-2 heavy backends warm on demand) → the

Connection card: endpoint, region, dummy creds, one-click copy-paste SDK/CLI/Terraform snippets. Ends "⌚ waiting for your first call."

Moment 2 — point the real SDK at it, unchanged → it works (*chord: confidence*)

First call arrives → card reacts live ("✓ first call · N services · all conformant") → **Conformance panel** is a live green wall, per service, tied to the real SDK version. Fidelity is demonstrated, not claimed.

Moment 3 — break something → glass-box → snapshot / fork / fix / replay (*chord: control*)

- **Inspect:** live, filterable call timeline; expand any call → request/response + plain-language *why it failed*.

- **Snapshot/fork:** git-like semantics over **real** backend state.
- **Fix + replay:** re-run the failed call; watch 404→200 with a before/after diff.

Build-&-run arc (the data-plane proof)

Spin compute + bucket + DB → **SSH into the real container** (web terminal) → deploy an app → it writes objects to **MinIO** and runs queries against **MySQL** → open the app's **URL**. The Inspector links `PutObject 200` → **the actual object bytes in MinIO**, `INSERT` → **the real row**. This shows the seam between cloud facade and real substrate — impossible on a black-box cloud.

6. Per-resource "Connect to the real thing"

The Connection card generalizes: every resource exposes **native access to its real backend**.

- **Compute (LXD/Docker):** `ssh -i vyomi.pem -p <port> ubuntu@host` + web terminal + the deployed app's reachable URL. *(Backed by the Docker/LXD SSH port-publish + routable-IP work.)*
- **S3 (MinIO):** object browser over the real bucket (upload/download real bytes), plus the S3 endpoint + SDK snippet; reachable by real S3 clients simultaneously.
- **RDS (MySQL/Postgres/PGLite):** a real SQL console + connection string / JDBC URL; DDL/DML persists. *(RDS Data API is the relay-safe variant.)*
- **Secrets/KMS (Vault), Queues (NATS/SQS):** same pattern — connect to the real engine.

7. Native look-and-feel: recognizable, not cloned (*anti-mimicry tenet*)

We **drop native look-and-feel as a goal, an architecture, and a success metric**. We **keep native familiarity as a cheap, optional lens**.

Keep (cheap, high value)	Drop (expensive, unwinnable, off-strategy)
Native terminology (bucket, instance, DB instance, IAM policy)	Pixel-perfect layout cloning
Native iconography + resource mental-model	Chasing console feature breadth
Native JSON/policy/config shapes (skills transfer)	"Indistinguishable from AWS" as the bar
Enough recognizability to orient a cloud-literate dev	Tracking hyperscaler console redesigns

Why: (a) the product-dev buyer works in IDE/CLI + glass-box — native L&F is low-value there; (b) a perfect AWS skin *hides* our differentiators (glass-box, CLI-reveal, snapshot/fork, connect-to-real-backend); (c) three hand-built skins are a treadmill we chose to leave.

How it stays cheap: because the console is **catalog/conformance-driven**, a "native lens" is **theming over the one engine** (labels, icons, grouping, resource shapes) — not a fork into three hand-built consoles.

Rule:

- **Default & primary:** the Vyomi-native, learning-first, glass-box, real-substrate console — this is our identity.
- **Optional lens:** an engine-themed "familiar (AWS/GCP/Azure)" mode — built **only** when a training/cert customer needs it, never up front, never a clone.
- **Never** measure success against "closeness to the native console." Measure it against **time-to-first-running-app, fidelity-proven, and glass-box comprehension.**

8. Architecture principle — console as a *view over conformance*

The console is a **thin, generated view driven by the conformance layer + catalog fixtures**

(`wasm/fixtures/{aws,gcp,azure}-catalog.json` , the same handlers that serve the SDK) — not a separate hand-built artifact.

- **Coverage grows for free:** each service proven in conformance appears in the console automatically.
- **One engine, optional skins:** the native lens (§7) is theming, not a fork.
- **Robustness comes from the backend** (the conformance suite), where it already lives.

9. Governance shell (what gets it *licensed*)

Same design system, a second entry for platform owners: **Fleet** (who runs what) · **Cost & TTL** (existing meter/reaper, surfaced) · **Profiles/policy** (allowed services per team) · **Identity/SSO/audit** · **air-gap/offline** toggle. Dev-love drives adoption; governance drives the signature.

10. Consequences to design for honestly

1. **Snapshot/fork is more powerful *and* heavier** — it captures real MinIO + SQL + container state. Show what's included and roughly how big.
2. **Real backends are heavier than mocks** — the "seconds to boot" promise is staged via Wave-1/Wave-2 progressive startup + lazy backend provisioning, with an honest warming status ("RDS: provisioning MySQL... 4s").

11. Sequencing (no big-bang)

1. Extract a **shared console engine** over the relay-proven `aws-core` services (S3, DynamoDB, RDS, IAM, KMS, Secrets, SQS/SNS).
2. Build the **persistent shell + Connection card + Conformance panel** (Moments 1–2).
3. Build the **glass-box Inspector** (Moment 3) — flagship; hardest to get right.
4. Add **per-resource Connect** surfaces (compute SSH, object browser, SQL console) — the data-plane proof.
5. Add **snapshot/fork/replay** over real state.
6. Keep current consoles running until the engine overtakes them; add the optional native lens only on customer demand.

Success metrics: time-to-first-running-app · % actions with CLI-reveal used · glass-box comprehension (can a dev self-diagnose a 4xx?) · conformance coverage — **not** console-to-native similarity.

12. Flagship spec — the glass-box Inspector

The Inspector is the surface no black-box cloud can offer, and the hardest to get right. It is a persistent right-hand drawer (`%I`) with four tabs.

12.1 The four tabs

Tab	Shows	Backed by
Calls	live, filterable timeline of every API call	request tap (§12.3)
State	browse the <i>real</i> backend state (objects, rows, containers, secrets)	per-resource backend readers
Snapshots	snapshot / fork / rollback tree over real state	§12.6
Conformance	per-service checks vs the real SDK	conformance suite

12.2 The captured-call event (data model)

Every intercepted request produces one immutable event (ring buffer, newest first):

```
{
  "id": "call_01H...",           // ULID, monotonic
  "ts": "2026-09-27T12:04:01.412Z",
  "cloud": "aws",                // lens
  "service": "s3",               // resolved from host/path/target
  "action": "HeadObject",        // SDK operation name
  "http": { "method": "HEAD", "path": "/acme/report.csv", "status": 404, "ms": 3 },
  "principal": "test",           // SigV4 access-key id (never the secret)
  "request": { "headers": {...redacted...}, "query": {...}, "body_summary": "..."},
  "response": { "code": "NoSuchKey", "headers": {...}, "body_summary": "..."},
  "backend": { "kind": "minio", "served_by": "s3_core", // which core/seam handled it
               "ref": { "type": "s3.object", "bucket": "acme", "key": "report.csv" } },
  "explain": { "level": "error", "why": "...", "hint": "...", "related": ["call_..."] },
  "snapshot_id": "snap_..."    // state generation this call ran against
}
```

The two fields that make it a *glass* box: `backend.served_by` (proves which real core/engine ran, not a mock) and `backend.ref` (the deep-link to the real resource — §12.4).

12.3 Capture mechanism

A single ASGI middleware taps every request/response in `server.py` (composes with the existing `_record_usage(...)` seam). It resolves `service / action` the same way the router does (host + path + `X-Amz-Target` + Query Action), stamps latency, redacts credentials (§12.7), and pushes onto an in-memory ring buffer (default 500 calls; configurable). The Inspector streams new events over SSE/WebSocket. Zero persistence by default — it's the user's own sandbox.

12.4 The call → real-backend link (the seam)

`backend.ref` is a typed handle the UI turns into a one-click deep-link into the **State** tab / resource detail — showing the *actual* effect of the call:

Call	backend.ref	Click-through shows
PutObject / HeadObject	{s3.object, bucket, key}	the real object bytes in MinIO
ExecuteStatement / INSERT	{rds.rows, db, table, pk?}	the real row(s) in MySQL/Postgres
RunInstances	{ec2.instance, id}	the real Docker/LXD container + SSH
GetSecretValue	{secrets.secret, name}	the real Vault entry (value masked)

This is the demo line "PutObject 200 → click → the actual bytes": the Inspector proves the control-plane call and the data-plane reality are the same thing.

12.5 The "why it failed" explainer

A rules engine over the event + recent related events produces plain-language cause + hint (no LLM needed for the common cases; deterministic and offline):

- Map error `code` → human explanation (`NoSuchKey` → "that key doesn't exist yet").
- **Cross-reference recent calls** to spot the *actual* mistake, e.g. a `404 NoSuchKey` on `HeadObject s3://acme/report.csv` when a recent `PutObject 200` wrote `s3://acme-reports/report.csv` → "key never written — your PutObject used bucket 'acme-reports', not 'acme'." + `related:` [that `PutObject`].
- Common families covered: auth/SigV4, missing resource, wrong region/endpoint, IAM deny (link to the evaluated policy), validation, throttling.
- Unknown cases degrade gracefully to code + doc link; never a dead end.

12.6 Snapshot / fork / rollback / replay over real backends

A **snapshot** is a manifest bundling a point-in-time capture of every backend **plus** the control-plane STATE:

Backend	Snapshot primitive
S3 / MinIO	bucket/object versioned copy (or data-dir tar)
RDS / MySQL·Postgres	logical dump (<code>mysqldump</code> / <code>pg_dump</code>) or data-dir snapshot; PGlite → serialize
Compute / Docker·LXD	<code>docker commit</code> + volume snapshot / <code>lxc snapshot</code>
Secrets·KMS·Queues	engine export
Control plane	the appliance STATE (resource metadata) at that instant

- **Snapshot** → new immutable generation; UI shows contents + approximate size.

- **Fork** → a new isolated workspace/namespace initialised from a snapshot (reuses the `core/shared_tenancy.py` namespace isolation). Two branches diverge independently.
- **Rollback** → restore backends + STATE to a snapshot generation.
- **Replay** → re-issue a captured call (or a range) against the current state; render a before/after **diff** (status + affected `backend.ref`).

Honest constraints (surface them in UI):

1. Real snapshots are heavier than mock state — show size/time; offer a **"control-plane only"** (fast, metadata) vs **"full data"** (real bytes/rows) mode.
2. Replaying **non-idempotent** calls has side effects — default replay onto a **fork**, and flag mutating calls before replay-in-place.
3. Big DB/volume forks are expensive — copy-on-write where the engine supports it; otherwise warn.

12.7 Performance, privacy, redaction

- **Overhead:** capture is O(1) per call (ring buffer, lazy body summarisation, cap body capture at N KB). Deep capture is opt-in per service.
- **Redaction:** never store the SigV4 secret, `Authorization`, KMS plaintext, secret values, passwords — redacted at tap time. Only the access-key `id` is kept (as `principal`).
- **Scope:** it's the user's own sandbox; nothing leaves the workspace. In the hosted (Codespaces) model the buffer stays in the tenant's instance.

12.8 UI interactions

- Filter by service/status/principal; free-text search; pause/resume stream.
- Click a row → detail pane (request/response, explain, `related` links, `[view in backend ▶]`, `[reveal CLI]`, `[snapshot]` `[fork]` `[replay]`).
- Keyboard: `⌘I` toggle · `/` filter · `j/k` move · `Enter` expand · `.` replay.

12.9 Build sequence (within §11 step 3)

1. Request tap + event schema + ring buffer + SSE stream + **Calls** tab.
2. `backend.ref` resolver + **State** deep-links (start with S3/MinIO + RDS).
3. Explainer rules (auth, missing-resource, IAM-deny, cross-reference).
4. Snapshot/rollback (control-plane first, then per-backend data).
5. Fork (namespace clone) + replay-on-fork + diff.

13. Per-resource "Connect to the real backend" surfaces

Where "you can actually build here" is won. Every resource detail follows **one common Connect contract**, so the experience is consistent across services and clouds.

13.1 The common Connect contract (shared layout)

```

└─ <resource-id> _____ [ Familiar ▾ ] [⋮ actions] ┘
└─ Connect _____┘
└─ <native access> _____ (endpoint / SSH cmd / connection string) ┘
└─ <native client> [sdk][cli][tf] <copy-paste snippet> ┘
└─ [ reveal CLI: <the exact provider command> ] ┘
└─ Live (real · <backend>) _____┘
└─ <the real backend's data/shell – objects / rows / terminal> ┘
└─ 🍷 backed by <MinIO | MySQL | Docker/LXD | Vault | NATS> ┘
└─ Actions _____┘
└─ <lifecycle: stop/reboot/terminate/delete> [ snapshot ] [ fork ] ┘

```

Three invariant blocks: **Connect** (native access + client snippet + CLI-reveal), **Live** (the real backend, in-line), **Actions** (lifecycle + snapshot/fork). The `backed by` badge is mandatory — it is the data-plane proof.

13.2 Compute (LXD / Docker) — SSH + web terminal + app URL

```

└─ i-0abc12 · running · t3.medium _____ [ Familiar ▾ ] [⋮] ┘
└─ Connect _____┘
└─ SSH ssh -i vyomi-i-0abc12.pem -p 12250 ubuntu@<host> ┘
└─ (Linux host → direct: ssh ubuntu@10.201.4.7 on :22) ┘
└─ [ ↓ download .pem ] [ ▶ open web terminal ] ┘
└─ App http://<host>:8080 (your deployed app) ┘
└─ [ reveal CLI: aws ec2 run-instances ... | gcloud compute ssh ... ] ┘
└─ Live (real · Docker/LXD) _____┘
└─ web terminal _____┘
└─ ubuntu@i-0abc12:~$ git clone ... && ./deploy.sh ┘
└─ _____┘
└─ procs: sshd, app:8080 · 🍷 backed by Docker/LXD ┘
└─ Actions _____┘
└─ [ stop ] [ reboot ] [ terminate ] [ snapshot ] [ fork ] ┘

```

- **SSH** reflects the two modes from the SSH work: routable-IP `ubuntu@<ip>:22` on a Linux host; port-publish `-p <host_port>` everywhere (incl. macOS Docker Desktop).
- **Web terminal** = in-browser SSH over WebSocket (build on the existing `ws_ec2_console` path), so "SSH in" works even without a local client.
- **App URL** = the deployed app's reachable published/forwarded port — the proof you *ran a product*, not just an instance.
- `.pem` download uses the existing `/api/.../private-key.pem` endpoints (Docker key fallback already added).

13.3 S3 (MinIO) — object browser over the real bucket

```

└─ acme-reports · bucket _____ [ Familiar ▾ ] [⋮] ┘
└─ Connect _____┘
└─ Endpoint https://<host>:9000 ┘
└─ [boto3][cli][tf] s3 = boto3.client("s3", endpoint_url=...) ┘
└─ [ reveal CLI: aws s3 cp ./report.csv s3://acme-reports/ ] ┘

```

```
| — Live (real · MinIO) ————— |
| 📁 / [ ↑ upload real file ] |
|   □ report.csv      2.4 KB   2026-09-27 [ view ▶ ] [ presign ] [🗑️] |
|   □ q1/             prefix |
|   → view: text | hex | image preview (real bytes from MinIO) |
|   🗑️ backed by MinIO · bucket "acme-reports" · reachable by real SDK |
| — Actions ————— |
| [ empty ] [ delete bucket ] [ snapshot ] [ fork ] |
```

- Real **upload/download/view** of bytes (text/hex/image preview), **presigned URL** generation, prefix navigation — all against the real MinIO bucket.
- Concurrently reachable by the user's real S3 SDK/CLI at the same endpoint (the point).

13.4 RDS (MySQL / Postgres / PGlite) — real SQL console

```
acme-db · db instance · available [ Familiar ▾ ] [...]
└─ Connect ────────────────────────────────────────────────────┐
| Host <host>:5432 DB acme User admin                        |
| JDBC jdbc:postgresql://<host>:5432/acme                    |
| [psycopg2][mysql][tf] conn = psycopg2.connect(...)         |
| (relay-safe: RDS Data API – ExecuteStatement)               |
└─ Live (real · PostgreSQL 18) ───────────────────────────────┘
| schema ▶ public.orders(id, sku, qty)                       |
|   SQL ────────────────────────────────────────────────────┐
| SELECT * FROM orders WHERE qty > 10;                      | [ ▶ run ] |
└───────────────────────────────────────────────────────────┘
id | sku    | qty    ← real rows, persist across the session
 1 | A-100  | 24
🌀 backed by PostgreSQL 18 (PGLite/MySQL swappable behind seam)
└─ Actions ────────────────────────────────────────────────────┘
[ reboot ] [ delete ]                                [ snapshot ] [ fork ]
```

- Real **SQL console** (DDL/DML persists) + **schema browser** + result grid.
- Connection string / JDBC for external clients; **RDS Data API** shown as the relay-safe path (survives the HTTP relay when a bare TCP socket can't).
- `snapshot` = logical dump; `fork` = a branched DB with the real data (test a migration, then roll back).

13.5 Shared behaviours

- **Snapshot/fork** on any resource routes to §12.6 (that resource's backend primitive).
- `[ Familiar ▼ ]` applies the optional native lens (§7) — theming only.
- Every Connect surface is **generated from the catalog/conformance definition** (§8); adding a conformant service yields its Connect surface with minimal bespoke UI (the three blocks + the backend reader).
- **backed-by badge is mandatory** — no resource detail ships without naming its real backend.

14. Effort & phased delivery plan

14.1 The effort driver: widgets, not services

~28 services across AWS/GCP/Azure collapse onto **9 reusable backend widgets**. Build each widget **once**, reuse across all three clouds; a service then costs only thin *wiring* (catalog map + `backend.ref` + connect snippet + snapshot primitive). So total cost = **foundation + 9 widgets + 28x thin wiring**, not 28 console builds.

Widget	Services it covers (AWS · GCP · Azure)	count
object-browser	S3 · GCS · Blob	3
sql-console	RDS · Cloud SQL · Azure SQL	3
compute-terminal	EC2 · GCE · Azure VM	3
serverless-invoke	Lambda · Cloud Functions · Azure Functions	3
nosql-item-viewer	DynamoDB · Firestore · Cosmos	3
kv/secret-viewer	Secrets Mgr · Secret Manager · Key Vault (secrets)	3
kms-crypto-view	KMS · Cloud KMS · Key Vault (keys)	3
queue/topic-viewer	SQS+SNS · Pub/Sub · Service Bus	4
generic-control-plane	IAM · VPC · API-GW · EventBridge/Eventarc · RBAC · VNet · APIM	~11

Sizing key: S ≈ 3–5 days · *M* ≈ 1–2 wk · *L* ≈ 2–3 wk · *XL* ≈ 3–4 wk (1 FE engineer; rough, engine-dependent — treat as relative).

14.2 Foundation (one-time, cross-cloud) — the bulk of the cost

#	Item	Size
F1	Console engine + persistent shell + catalog-driven renderer (left rail, status bar)	L
F2	Connect-contract framework (the 3-block resource detail scaffold)	M
F3	Glass-box Inspector — Calls tab + capture middleware + event schema + SSE stream	L
F4	Inspector — <code>backend.ref</code> resolver + State deep-links + explainer rules	L
F5	Conformance panel (live per-service, from the test suite)	M
F6	Snapshot/rollback engine (control-plane + per-backend primitives)	L
F7	Fork (namespace clone) + replay-on-fork + diff	L
F8	CLI/SDK/IaC reveal framework	M
F9	Native-lens theming layer (<i>optional, deferred</i>)	M

Foundation $\approx$ **20–25 eng-weeks** (F6/F7/F9 can be phased later).

14.3 Widgets (reusable across clouds)

Widget	Size	Notes
object-browser	M	upload/download/view bytes, presign, prefixes
sql-console	L	query editor + result grid + schema browser
compute-terminal	L	web SSH + connect card + app URL (reuses <code>ws_ec2_console</code> + the SSH work)
serverless-invoke	M	deploy/invoke/logs
nosql-item-viewer	M	table/collection browse, item CRUD, query
kv/secret-viewer	S–M	list, versions, masked reveal
kms-crypto-view	S–M	keys + encrypt/decrypt/sign playground
queue/topic-viewer	M	send/receive/peek, subscriptions, fan-out
generic-control-plane	M	auto-form list/detail from catalog + IAM policy-simulator view

Widgets $\approx$ **18–20 eng-weeks**. Per-service wiring $\approx$ **3–4 eng-weeks** total ($28 \times \sim S/2$).

14.4 Per-cloud service $\rightarrow$ widget $\rightarrow$ wiring

AWS (relay-proven core): S3 $\rightarrow$ object · RDS $\rightarrow$ sql · EC2 $\rightarrow$ terminal · Lambda $\rightarrow$ serverless · DynamoDB $\rightarrow$ nosql · SQS/SNS $\rightarrow$ queue · KMS $\rightarrow$ kms · Secrets $\rightarrow$ kv · IAM/VPC/API-GW/EventBridge $\rightarrow$ generic. **GCP**: Storage $\rightarrow$ object · Cloud SQL $\rightarrow$ sql · GCE $\rightarrow$ terminal · Functions $\rightarrow$ serverless · Firestore $\rightarrow$ nosql · Pub/Sub $\rightarrow$ queue · Cloud KMS $\rightarrow$ kms · Secret Mgr $\rightarrow$ kv · IAM/VPC/API-GW/Eventarc $\rightarrow$ generic. **Azure**: Blob $\rightarrow$ object · SQL $\rightarrow$ sql · VM $\rightarrow$ terminal · Functions $\rightarrow$ serverless · Cosmos $\rightarrow$ nosql · Service Bus $\rightarrow$ queue · KV keys $\rightarrow$ kms · KV secrets $\rightarrow$ kv · RBAC/VNet/APIM $\rightarrow$ generic.

Each cell after its widget exists = **thin wiring (S/2)**: catalog map + connect snippet

- CLI-reveal string + `backend.ref` + snapshot primitive.

14.5 Honest caveats (temper the "all green" inventory)

- **Rich widget is conformance-gated.** A service earns its data-plane widget only when its conformance is green; otherwise it falls back to the **generic-control-plane** view. The conformance panel (F5) is the honest gate — no shallow-parity screens.
- **gRPC-over-relay limit:** Firestore / Pub/Sub are REST-proven; their in-console widgets work, but *external* gRPC clients won't traverse the HTTP relay. Surface this in the Connect block, don't hide it.
- **Operation-level depth varies** (VPC peering, API-GW authorizers, Cosmos query, GCS list, S3 checksum have historically been thinner). Widgets cover the common path; the conformance panel shows exactly what's verified.

14.6 Phases

Phase	Scope	Deliverable	~Duration (1 FE)
P0 — Foundation + 1 vertical	F1,F2,F3,F8 + object-browser + wire S3	shell + glass-box Calls + S3 Connect over real MinIO; the 3-chord demo for one service	8–10 wk
P1 — AWS data-plane (biggest payoff)	F4,F5 + sql/terminal/serverless/nosql/queue/kv/kms/generic widgets + wire all AWS	full AWS console redesign, glass-box, connect-to-real-backend, conformance-gated	12–15 wk
P2 — Reproducibility	F6,F7	snapshot / fork / rollback / replay over real backends	5–6 wk
P3 — GCP	reuse all widgets; wire GCP services (+ GCP snippets/CLI/terminology)	full GCP console (mostly wiring)	4–6 wk
P4 — Azure	reuse all widgets; wire Azure services	full Azure console (mostly wiring)	4–6 wk
P5 — Lens + depth + polish	F9 native lens (on demand) · serverless deploy depth · governance shell polish	optional familiar-mode + op-level depth	4–6 wk

14.7 Totals & scheduling

- **Full 3-cloud redesign ≈ 38–49 eng-weeks.**
- Front-loaded: **P0+P1 (≈ 20–25 wk) delivers the entire AWS experience + the demo** — the licensable story lands before GCP/Azure (which are then cheap wiring).
- Team scaling (rough): **~9–12 months** at 1 FE · **~5–6 months** at 2 FE · **~3–4 months** at a 3-person squad.
- **De-risking order:** prove the engine + glass-box + one real-backend widget in P0 before committing to the widget fleet. If P0 lands the demo, the rest is repetition.

14.8 Substrates, one engine — Nano included

The console is a client SPA over `/api/*`. **Who serves that API is the substrate** — and the engine doesn't care. This is the "not a fork" (ADR-001) principle made concrete:

Substrate	API served by	Data plane	Real compute
Local <code>vyomi up</code>	FastAPI <code>server.py</code>	Docker MinIO/MySQL, LXD/Docker	✓
Codespaces sandbox	FastAPI (in Codespace)	same, forwarded	✓
Nano (browser)	Service Worker → Pyodide cores	in-WASM stores (ObjectStore, PGlite Postgres, sqlite, Vault-equiv, NATS-equiv)	✗

So Nano is **not another console** — it's the same engine, same widgets, pointed at the SW-served API. External SDK/CLI clients reach Nano via the **relay** (already built).

14.9 Nano capability matrix (widgets on WASM)

Most widgets work on Nano because the backends have WASM equivalents; the exception is **real compute** (no LXD/Docker in a browser tab).

Widget	Nano?	Nano backing
object-browser	✓	in-WASM ObjectStore (OPFS/in-mem)
sql-console	✓	PGlite (real Postgres in WASM) / sqlite
nosql-item-viewer	✓	in-WASM NoSqlStore
kv/secret-viewer	✓	in-WASM KvStore
kms-crypto-view	✓	stdlib AEAD crypto core
queue/topic-viewer	✓	in-WASM MessagingStore (SQS/SNS fan-out)
generic-control-plane	✓	in-proc policy engine (IAM eval)
compute-terminal	⚠ degraded	no VM/SSH → metadata + "open a Codespace to SSH" CTA
serverless-invoke	⚠ partial	invoke works (Pyodide); no container deploy

Scope note: Nano's *rich* data-plane is the **AWS-core** set; GCP/Azure on Nano fall back to the generic-control-plane (console-CRUD) view, consistent with today.

Nano is actually the *best* substrate for two flagship features:

- **Glass-box capture is easier** — all calls flow through one in-tab SW/router, so the Inspector taps a single JS/Pyodide seam (vs the server middleware on real substrates).
- **Snapshot/fork is lighter** — WASM store state is serializable in-process (PGlite dump / OPFS / JSON), so branch/rollback is cheaper than snapshotting real MinIO+MySQL.

14.10 Nano effort delta & where it slots

Because widgets are reused and the engine is substrate-agnostic, Nano is **not** a second build — it's a cross-cutting workstream (mostly one-time), *provided the engine is designed substrate-agnostic from P0*:

Item	Size
Substrate-capability model (per-substrate widget/service availability + graceful degrade)	M
Glass-box capture — Service-Worker/router interceptor feeding the same event schema	M
Nano bundle integration (serve the engine in the SW bundle, base-path-aware; retire the old nano consoles)	M–L
Nano Connect variant (relay endpoint + SDK snippet instead of SSH/host:port)	S
Nano snapshot/fork (serialize WASM stores / PGLite dump)	M
Compute/serverless capability-degrade UI ("run this in a Codespace")	S

Nano delta $\approx$ 5–7 eng-weeks, one-time. Slots as **P-Nano** after P1 (once the AWS widgets exist, Nano is: wire engine into the SW bundle + capability-gate compute + Nano Connect/snapshot). Design constraint on P0: **the engine must talk only to `/api/*` and carry no substrate assumptions**, so Nano stays free-riding.

Updated totals: full 3-cloud $\times$ 3-substrate redesign $\approx$ **43–56 eng-weeks** (the +5–7 is the Nano cross-cutting delta; GCP/Azure on Nano ride the generic widget).

15. UI commonality guarantee (HARD CONSTRAINT)

The console UI must be identical across local-install, Codespaces sandbox, and Nano. This is not a goal — it is an enforced invariant. Substrate differences are expressed as *data* (a capability manifest), never as forked UI.

15.1 The six guarantees that make it true

1. **One console artifact.** A single SPA bundle, built once, shipped to all three substrates. Local/Codespaces serve it as static assets from FastAPI; Nano serves the *same* bundle via its Service Worker. No copies, no per-substrate builds.
2. **Substrate-agnostic API contract.** The SPA talks **only** to `/api/*`. Who serves it — FastAPI (`server.py`) or the SW→Pyodide router — is transparent to the UI. **No `if (nano)` / `if (local)` branching in UI code.**
3. **Differences are a capability manifest, not screens.** The UI reads one runtime descriptor and gates/degrades from it:

```
GET /api/console/capabilities
{
  "substrate": "nano",           // local | codespaces | nano
  "cloud_lenses": ["aws"],      // lenses with a rich data-plane here
  "widgets": { "object-browser":"full", "sql-console":"full",
               "compute-terminal":"degraded", "serverless-invoke":"partial" },
  "features": { "snapshot":true, "fork":true, "relay":true, "ssh":false },
```

```
"connect": { "mode":"relay" }    // relay | endpoint | ssh
}
```

Every substrate difference (Nano has no real compute; local exposes SSH; Codespaces forwards a URL) is one field here. The **compute-terminal component is the same everywhere** — it reads `widgets["compute-terminal"] == "degraded"` and shows the "open a Codespace to SSH" state; it is never a different screen.

4. **One component tree.** Same shell, left rail, status bar, Inspector, Connect contract, widgets — everywhere. Only capability-driven content differs.
5. **One glass-box event schema.** The two capture implementations (FastAPI middleware vs SW/router interceptor) emit the **identical** event shape (§12.2), so the Inspector UI is byte-for-byte the same across substrates.
6. **Serving-agnostic / base-path-aware.** The bundle works at web root *and* under a subpath (Nano's `/nano/` SW scope) — already solved in the Nano bundle; the new engine inherits it.

15.2 The anti-fork rule

- Substrate/cloud variation is absorbed by **capability flags + widget config**, never by a copied component. The moment someone writes "the Nano object-browser," the guarantee is broken.
- The **Connect block** is one component with a `connect.mode` variant (`ssh / endpoint / relay`) — *not* three Connect components.
- This is the *recognizable-not-cloned* discipline (§7) applied to substrates: accommodate by parameter, never by fork.

15.3 Enforcement (so it can't drift)

A **cross-substrate parity test in CI** — the same "no false green" ethos as the conformance suite:

- Boot the console against all three substrates (or their capability manifests) and assert the **same component tree renders** and the **same journeys pass**, with only the documented degradations differing.
- A lint rule fails the build on substrate-name branching (`if (substrate === ...)`) in UI components — force it through the capability layer.
- The demo journeys (Moments 1–3, build-&-run) run on **all three substrates** in CI; a screen that exists on one but not another is a build failure.

Net: commonality is guaranteed by construction (one bundle + `/api/*` + capability manifest) and protected by CI (parity test + no-substrate-branch lint) — it cannot silently diverge.

16. Tech stack decision (hard to change later)

Chosen: Lit (Web Components) + design tokens (CSS custom properties).

Selected against four requirements — enterprise-grade, cross-resolution, browser-native (Nano), and white-label-ready:

- **White-label (decisive):** Shadow DOM = collision-free style encapsulation; CSS custom properties = per-tenant theming (logo/palette/type) with no code change. The console embeds into a customer's own portal as `<vyomi-console theme="acme">` — the literal white-label use case. Framework SPAs can't embed into arbitrary hosts without CSS bleed the way Shadow-DOM components can.
- **Enterprise-grade / durability:** Web Components are the W3C standard — framework-agnostic and outlive framework churn (no rewrite when React→next). The tech serious enterprise design systems ship on (Microsoft Fluent UI WC, Adobe Spectrum, SAP UI5, Shoelace). Lit is Google-backed and mature.
- **Browser / Nano:** ~5 KB runtime — serves cleanly from the Nano Service Worker, is base-path-friendly, and doesn't fight Pyodide for the tab's weight budget.
- **Cross-resolution:** CSS-driven responsiveness (tokens + container queries), no framework constraint.

Implementation rules:

- **No mandatory build step** — ES modules; **vendor Lit** (not a runtime CDN dep) so it works offline / air-gapped / under the Nano `/nano/` base path.
- **Accessibility base:** stand on an accessible WC library (Shoelace or Fluent WC) for primitives rather than hand-rolling a11y.
- **Interop:** WC are host-agnostic — embeddable inside a React/Vue customer portal later (white-label) without a rewrite.
- Tradeoff (smaller ecosystem than React) is mitigated by interop + the accessible base.